



Chapter S1D: Stack

CONTENT

No.	Topic	Syllabus Guide
1	Data Structure	1.3.1 – 1.3.2
	1.1 Static	
	1.2 Dynamic	
2	Introduction to Stack	1.3.3
3	Stack Implementation	
4	Stack Overflow and Underflow	
5	Stack Handling Algorithms	
6	Stack Manipulation	
7	Array Implementation of Stack	
8	Linked List Implementation of Stack	
9	Python List Implementation	
10	Use of Stack	
	10.1 In recursion	2.2.7
	10.2 To evaluate an arithmetic expression	
11	Tutorial	
12	Programming Exercise	

1. DATA STRUCTURE

A data structure is a group of related data items organised in the computer. It allows a large number of pieces of data to be managed as a single set. Each type of data structure makes it easy for the programmer to find and process the data in particular ways. Each type of structure has its own strengths and weaknesses. Examples of data structures are arrays, stacks, queues, linked lists, binary search trees, and hash tables.

1.1 Static

Static data structures do not change in size while the program is running. A typical static data structure is an array. Once you declare its size (or upper bound), it cannot be changed.

1.2 Dynamic

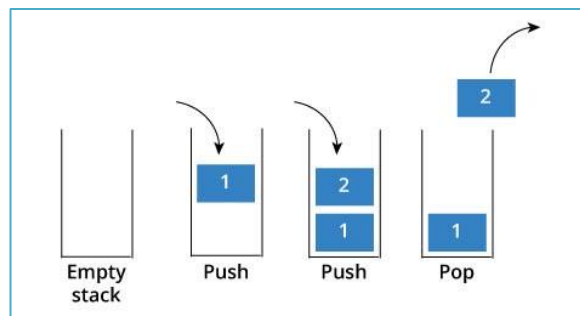
Dynamic data structures can increase and decrease in size while a program is running. A typical dynamic data structure is a linked list.

Type	Advantages	Disadvantages
Static	• Space allocated during compilation • Easier to program • Easier to check for overflow • Random access	• Space allocation is fixed • Wastes space when partially used
Dynamic	• Only uses space required • Efficient use of memory • Emptied storage can be returned to the system	• Difficult to program • Searches can be slow • Serial access

2. INTRODUCTION TO STACK

A **stack** is a collection of data items which can only be accessed at one end (i.e. at the top of the stack). Stacks are frequently used for temporary storage of data. Only two operations may be carried out on a stack:

- Adding a new item, called **PUSH**ing or stacking the item, involves placing it on the top of the stack.
- Removing an item, called **POP**ping the stack involves removing the item at the top of the stack.



A stack is considered a **last in, first out (LIFO)** data structure. A LIFO data structure is like a stack of plates. The last or most recent plate pushed (put) onto the stack is the first plate popped (removed) from the stack.

Another example of a LIFO structure is a text editor. As you type in characters, the character insertion pointer resides at the end of the last character typed in. If you press the backspace key the last character typed is removed.

The first value that is popped from the stack is the value that has remained on the stack for the shortest amount of time. The very last item popped from the stack will have remained on the stack for the longest time.

3. STACK IMPLEMENTATION

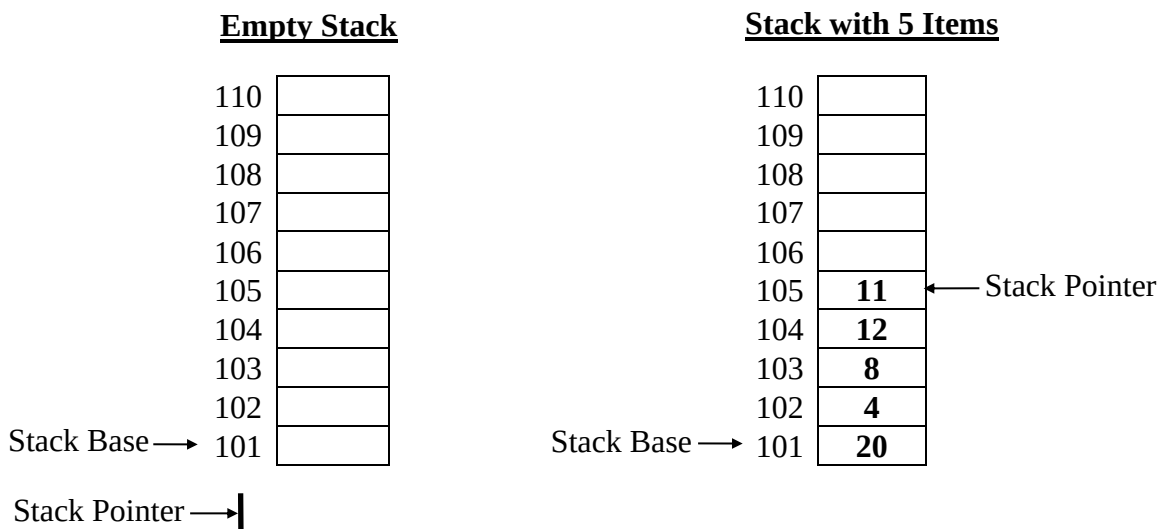
A block of memory, large enough to accommodate the stack at its maximum size, is reserved. Determining the size of this block can often be a critical design decision. If too little room is reserved, the stack will ultimately exceed the allotted storage space; however, if too much room is reserved, memory space will be wasted.

In a computer memory, a stack is indicated by 2 pointers:

- The **Stack Pointer** indicates the position of the top of the stack. The value of the stack pointer varies as data is pushed or popped from the stack.
- The **Stack Base** indicates the base of the stack. Its value is fixed as long as the stack existed.

The stack is empty if the value of the stack pointer is null.

The stack can be represented in a *logical view* as follows:



4. STACK OVERFLOW AND UNDERFLOW

A stack will always have a maximum size, because memory cannot grow indefinitely.

If the stack is **implemented as an array**, a full stack can be tested for by examining the value of the stack pointer.

- An attempt to push another item onto the stack would cause **overflow** so an error message can be given to the user.
- Similarly, if the stack pointer is null, the stack is empty and **underflow** will occur if an attempt is made to pop an item.

5. STACK HANDLING ALGORITHMS

Algorithm 1 – Setting up a stack

- Step 1. Reserve N spaces for the stack
- Step 2. Let (Stack Pointer) = null
- Step 3. Let (Stack Base) = Smallest location allocated
- Step 4. Exit

Example: An empty STACK with 10 locations

LOCATION	101	102	103	104	105	106	107	108	109	110
CONTENT										

Stack Base $\uparrow$ Stack Pointer $\rightarrow$

Algorithm 2 – Pushing an item onto a stack

- Step 1. If (Stack Pointer) = largest location allocated for stack then print "STACK FULL" and exit
- Step 2. If (Stack Pointer) = null, then set (Stack Pointer) = (Stack Base) and store the given item in location pointed to by (Stack Pointer)
- Step 3. Else increase (Stack Pointer) by 1, and store the given item in location pointed to by (Stack Pointer)
- Step 4. Exit

PROCEDURE PUSH

```

IF StackPointer = Limit THEN
    OUTPUT "STACK FULL" and EXIT
ENDIF
IF StackPointer = Null THEN
    StackPointer  $\leftarrow$  StackBase
    Stack[StackPointer]  $\leftarrow$  NewItem
ELSE
    StackPointer  $\leftarrow$  StackPointer + 1
    Stack[StackPointer]  $\leftarrow$  NewItem
ENDIF
ENDPROCEDURE

```

Example: Insert item [99] into the stack

Before:

LOCATION	101	102	103	104	105	106	107	108	109	110
CONTENT	20	4	8	12	11					

StackBase $\uparrow$ StackPointer $\uparrow$

After:

LOCATION	101	102	103	104	105	106	107	108	109	110
CONTENT	20	4	8	12	11	99				

StackBase $\uparrow$ StackPointer $\uparrow$

Algorithm 3 – Popping an item from a stack

- Step 1. If (Stack Pointer) = null, then print "EMPTY STACK" and exit
- Step 2. Else read content of location pointed to by (Stack Pointer)
- Step 3. If (Stack Base) not equal to (Stack Pointer) then decrease (Stack Pointer) by 1
- Step 4. Else set (Stack Pointer) = null
- Step 5. Exit

```

FUNCTION POP
    IF StackPointer = Null THEN
        OUTPUT "EMPTY STACK" and EXIT
    ELSE
        PoppedItem  $\leftarrow$  Stack[StackPointer]
    ENDIF
    IF StackBase  $\neq$  StackPointer THEN
        StackPointer  $\leftarrow$  StackPointer - 1
    ELSE
        StackPointer  $\leftarrow$  Null
    ENDIF
    RETURN PoppedItem
ENDFUNCTION

```

Example: Delete an item [99] from the stack

Before:

LOCATION	101	102	103	104	105	106	107	108	109	110
CONTENT	20	4	8	12	11	99				

$\text{StackBase} \nearrow$

$\nwarrow \text{StackPointer}$

After:

LOCATION	101	102	103	104	105	106	107	108	109	110
CONTENT	20	4	8	12	11					

StackBase StackPointer

6. STACK MANIPULATION

Example 1: Show the contents of the variables & the stack after program segment execution.

```

Initialise (s);
Push ('a', s);
Push ('b', s);
Push ('c', s);
Pop (x, s);
Pop (y, s);
Push (x, s);
Push (y, s);
Pop (z, s);

```

Solution:

STEP	STACK CONTENT
(1)	— / — / — / — / —
(2)	'a', — / — / — / — / —
(3)	'a', 'b', — / — / — / — / —
(4)	'a', 'b', 'c', — / — / — / — / —
(5)	'a', 'b', — / — / — / — / — (x='c') where x is a variable
(6)	'a', — / — / — / — / — (x='c') (y='b')
(7)	'a', 'c', — / — / — / — / — (x='c') (y='b')
(8)	'a', 'c', 'b', — / — / — / — / — (x='c') (y='b')
(9)	'a', 'c', — / — / — / — / — (x='c') (y='b') (z='b')

Example 2: A Stack has 6 memory cells with Top-of-Stack (TOS) = 3

STACK: 5, 2, 3, __, __, __

Find the output of the program segment:

- (a) Call POP (STACK, ITEMA);
 Call POP (STACK, ITEMB);
 Call PUSH (STACK, ITEMB + 2);
 Call PUSH (STACK, 8);
 Call PUSH (STACK, ITEMA + ITEMB);
- (b) Repeat
 Call POP (STACK, ITEM);
 Write ITEM;
 Until TOS = 0

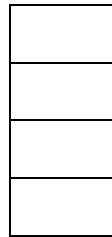
Solution:

(a) STEP	STACK CONTENT	
(1)	5, 2, 3, __, __, __	
(2)	5, 2, __, __, __, __	ITEMA = 3
(3)	5, __, __, __, __, __	ITEMB = 2
(4)	5, 4, __, __, __, __	
(5)	5, 4, 8, __, __, __	
(6)	5, 4, 8, 5, __, __	

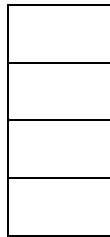
- (b) The output from stack is 5,8,4,5

Example 3: A stack with 4 locations is set up. Insert the Stack Base Pointer, Stack Pointer, the content of the stack and error messages (if any) in the above diagrams after the following steps are performed:

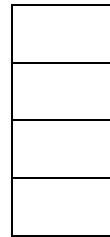
- | | |
|----------------------|---------------|
| (a) Set up the stack | (b) Push [50] |
| (c) Pop | (d) Push [35] |
| (e) Push [40] | (f) Push [80] |
| (g) Push [10] | (h) Pop |



(a)



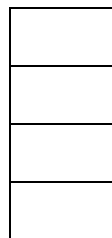
(b)



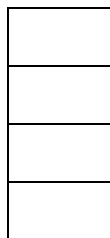
(c)



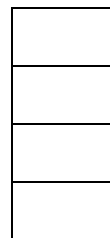
(d)



(e)



(f)



(g)



(h)

Example 4: A computer has a stack with data items entering the stack via the input stream and leaving the stack via the output stream. The only operations available are:

PUSH: push the next item in the input stream onto the stack

POP: pop the top item off the stack and put it in the output stream

Given the input stream ABCDE which of the following are possible output streams? For the possible output streams show, using the instructions **PUSH** and **POP**, how they can be obtained.

- (a) ACEDB
- (b) BCEAD
- (c) CBAED
- (d) EACBD

7. ARRAY IMPLEMENTATION OF STACK

A stack may be implemented as either a **static** or **dynamic** data structure. A static data structure such as an **array** can be used to implement a stack.

In an array implementation of a stack,

- stack base is at index 0, since it always point to the first location,
- stack pointer is used to indicate the top of the stack and
- a variable is used to hold the maximum size of the array.

The array implementation can be simplified to use only two variables:

- **top**: indicate top of the stack
- **maxSize**: to hold the size of the array (the maximum size of stack)

Example: Empty Stack

Index	0	1	2	3	4	5	6	7	8	9
Content										

top = -1 (null)

maxSize = 10

Example: Stack with 5 Items

Index	0	1	2	3	4	5	6	7	8	9
Content	20	4	8	12	11					

top = 4

maxSize = 10

The following operations can be used to implement a stack using an array.

- **push(item)** adds a new item to the top of the stack
- **pop()** removes and returns the top item from the stack
- **peek()** returns the top item from the stack but does not remove it
- **isEmpty()** tests to see whether the stack is empty and returns a Boolean
- **isFull()** tests to see whether the stack is full and returns a Boolean

The following pseudocode implements the stack operations using an array.

```

FUNCTION isEmpty
    IF top = -1 THEN
        RETURN True
    ELSE
        RETURN False
    ENDIF
ENDFUNCTION

```

```

FUNCTION isFull
    IF top = maxSize - 1 THEN
        RETURN True
    ELSE
        RETURN False
    ENDIF
ENDFUNCTION

```



```
PROCEDURE push(item)
  IF isFull THEN
    print ("Stack is full")
  ELSE
    top ← top + 1
    stack[top] ← item
  ENDIF
ENDPROCEDURE
```

```
FUNCTION pop
  IF isEmpty THEN
    print ("Stack is empty")
  ELSE
    item ← stack[top]
    top ← top - 1
    RETURN item
  ENDIF
ENDFUNCTION
```

```
FUNCTION peek
  IF isEmpty THEN
    print("Stack is empty")
  ELSE
    item ← stack[top]
    RETURN item
  ENDIF
ENDFUNCTION
```

8. LINKED LIST IMPLEMENTATION OF STACK

A *dynamic* data structure such as a linked list can be used to implement a stack.

In using a linked list to implement a stack,

- A top pointer, which is the Start pointer, is used to keep track of the top of the stack.
- Pushing and popping of items happens at the top pointer.

Using linked list over an array to implement a stack means that a stack can shrink or grow as much as needed. Since each new node is dynamically allocated, overflow is unlikely to happen.

The following operations can be used to implement a stack using a linked list.

- **push(item)** inserts item into linked list at top pointer
- **pop()** returns top element in linked list and moves top pointer to next node
- **peek()** returns top element without removing any node
- **isEmpty()** tests whether linked list is empty and returns Boolean

9. STACK – PYTHON LIST IMPLEMENTATION

The Python built-in `list` provides a simple way to implement a stack.

The **top** of the stack is the **last element** in the list.

Key methods:

- `len(stack)` determine number of items in the stack
- `append(item)` adds an item to the top of the stack (push)
- `pop()` removes and returns the top item of the stack
- `[-1]` access the top item of the stack without removing it

10. USE OF STACK

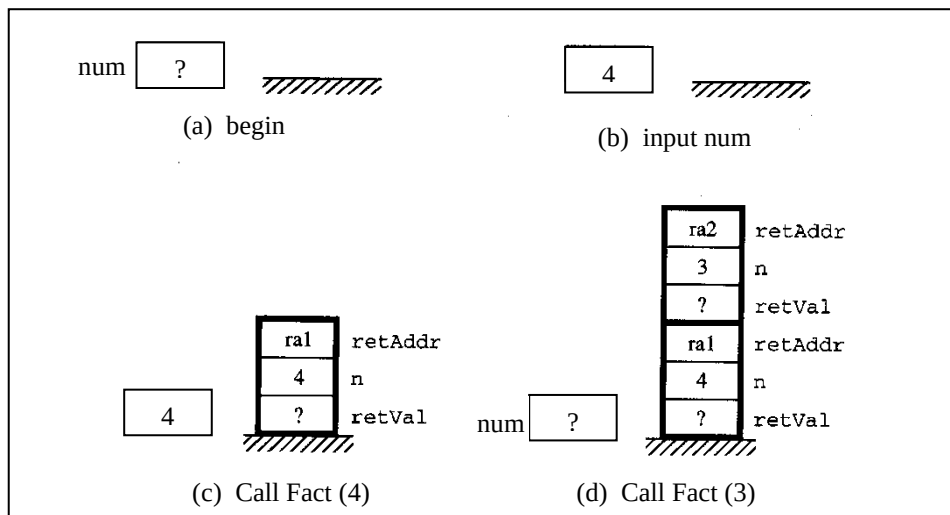
10.1 IN RECURSION

A recursive function (in Python) is to compute the factorial of an input number.

```
def Fact(n=int):
    if (n <= 1):
        return 1
    else:
        return n * Fact(n - 1)           # ra2

def main():
    num=int(input("Enter a small integer: "))
    print("Its factorial is:", Fact(num)) # ra1

main()
```



4 steps of tracing the recursive program to find the factorial of a given number 4 are shown. The data structure used in step (c) when the recursive function is called is a *call stack*.

A recursive function

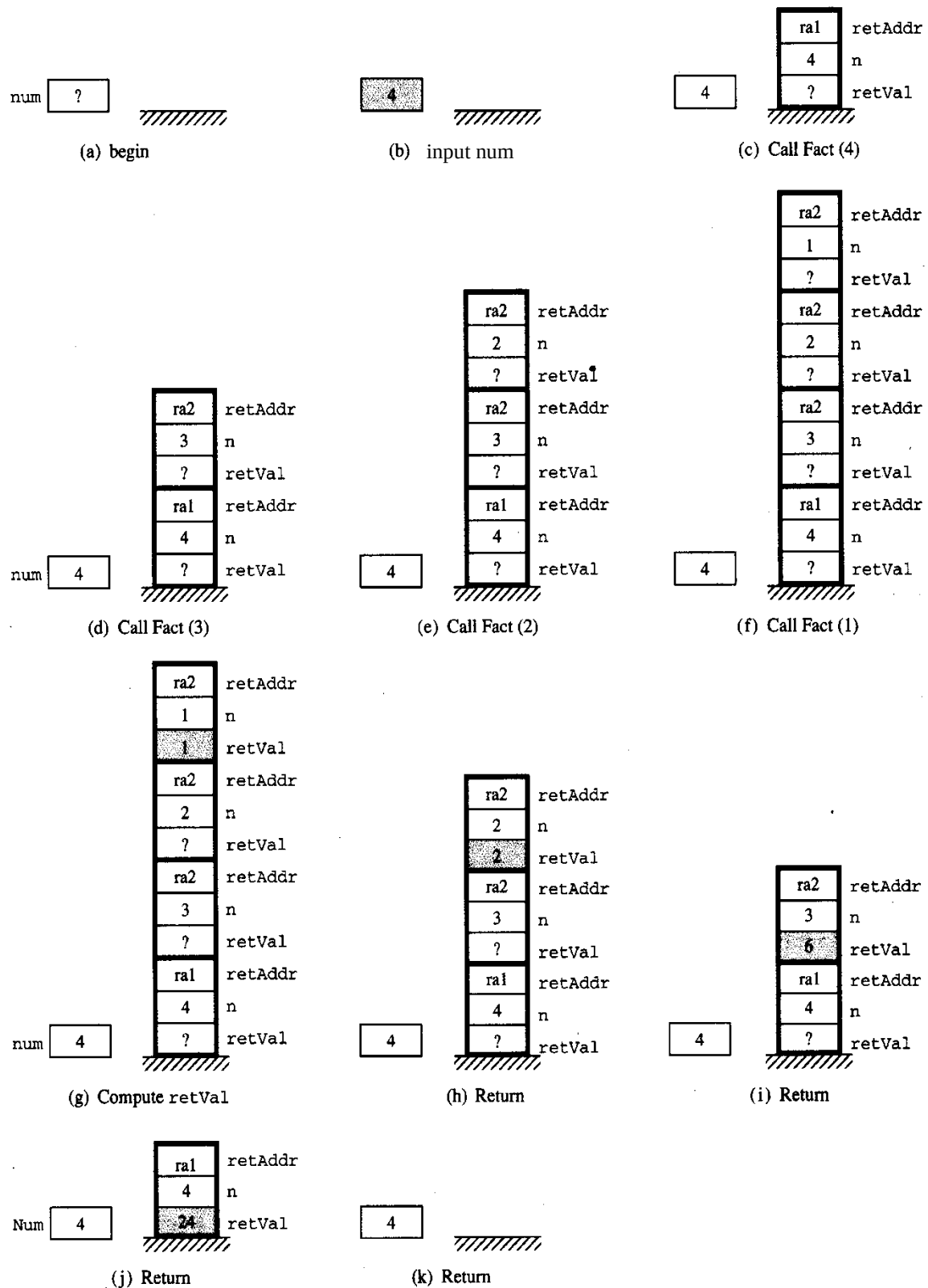
Recursive Definition

$f(n) = n.f(n-1)$ for $n > 1$

$f(1) = 1$

Factorial Function

$$\begin{aligned} f(4) &= 4f(3) \\ &= 4(3)f(2) \\ &= 4(3)(2)f(1) \\ &= 4(3)(2)(1) = 24 \end{aligned}$$



As each function call is called, the current state of the machine is saved so that the program can return correctly to the previous invocation. The return address, input parameter, local variables are saved in a stack frame and pushed onto the call stack. As each function call is completed, the stack frame is popped from the *call stack* and data is retrieved and the program continues the next instruction as indicated by the return address.

10.2 TO EVALUATE AN ARITHMETIC EXPRESSION

A+B Infix Notation

+AB Prefix Notation (Polish Notation)

AB+ Postfix Notation or Suffix Notation or (Reverse Polish Notation)

Examples:

Infix Notation	Prefix Notation	Postfix Notation
(A+B) *C	*+ABC	AB+C*
A+ (B*C)	+A*BC	ABC*+

Stacks are especially useful when evaluating arithmetic expressions. Computers typically convert **infix expressions** (e.g. **A+B**) to **postfix (AB+)** to eliminate ambiguity and simplify evaluation by avoiding the need for parentheses.

Computers evaluate arithmetic expressions more efficiently using **postfix notation**.

(a) Convert Infix notation to Postfix notation

Input String Q: 5 * (6 + 2) – 12 / 4 (Infix)

Output String P: 5 6 2 + * 12 4 / – (Postfix)

Algorithm

Step 1: PUSH a left parenthesis '(' onto STACK and append right parenthesis ')' to end of input string, Q

Step 2: While STACK is not empty, read Q from left to right and do steps 2a to 2d

Step 2a: If current character is an operand, copy to output string, P

Step 2b: If current character is a '(', PUSH it on STACK

Step 2c: If current character is an operator @, then

- POP operators (if any) from STACK while they have equal or higher precedence than @, and insert the popped operators in P
- PUSH @ to STACK

Step 2d: If current character is ')', then

- POP operators from STACK and insert in P until a '(' is at the top of STACK
- POP (and discard) the '(' from the STACK

Step 3: Exit

Initially Q: 5*(6+2) – 12 / 4

P: empty

	<u>SYMBOL ON Q</u>	<u>STACK CONTENT</u>	<u>EXPRESSION FOR P</u>
(1)	5	(	5
(2)	*	(*	5
(3)	(	(*(	5
(4)	6	(*(	5,6
(5)	+	(*(+	5,6
(6)	2	(*(+	5,6,2
(7)	)	(*	5,6,2,+
(8)	–	(–	5,6,2,+,*
(9)	12	(–	5,6,2,+,*,12
(10)	/	(–/	5,6,2,+,*,12
(11)	4	(–/	5,6,2,+,*,12,4
(12)	)	empty	5,6,2,+,*,12,4,/, –

Finally Q: 5*(6+2) – 12 / 4

P: 5,6,2,+,*,12,4,/, –

Practice: Convert the input string (Infix) $(6+2)*5-8/4$ to Postfix.

SYMBOL ON Q STACK CONTENT EXPRESSION FOR P

(1)

(2)

(3)

(4)

(5)

(6)

(7)

(8)

(9)

(10)

(11)

(12)

	Initially	Finally
Input String, Q	$(6+2)*5-8/4$	
Output String, P	empty	

(b) Evaluate a Postfix notation

Given Postfix string in stack P: 5 6 2 + * 12 4 / –

Algorithm

- Step 1: Append NULL character at end of P (*Processing stops when NULL is encountered*)
- Step 2: While NULL has not been encountered, read P from left to right and repeat steps 2a and 2b
- Step 2a: If current character is an operand, PUSH it onto STACK
- Step 2b: If current character is an operator @, then
Pop top 2 elements of STACK into variables x and y
Calculate y @ x
PUSH result to STACK
- Step 3: When NULL is encountered, POP top value of STACK (This is the result of the Postfix expression)
- Step 4: Exit

	<u>SYMBOL ON P</u>	<u>STACK CONTENT</u>
(1)	5	5
(2)	6	5,6
(3)	2	5,6,2
(4)	+	5,8
(5)	*	40
(6)	12	40,12
(7)	4	40,12,4
(8)	/	40,3
(9)	–	37

The evaluated value = 37

Practice: Evaluate the Postfix expression from previous page.

	<u>SYMBOL ON P</u>	<u>STACK CONTENT</u>
(1)		
(2)		
(3)		
(4)		
(5)		
(6)		
(7)		
(8)		
(9)		

The evaluated value =

11. TUTORIAL

1. A stack is a Last In, First Out (LIFO) data structure. It has a pointer called **top** and the operations **push** and **pop**.
 - (a) What is meant by a LIFO data structure?
 - (b) Name and describe two types of errors that could occur when attempting to add a data item or remove a data item from the stack.
 - (c) Describe the operations push and pop.
 - (d) Write pseudocodes to implement the push and pop operations using a fixed size array.
 - (e) Besides push and pop, other operations required to implement stack as an array are peek, isEmpty and isFull. Fill in the table when the operations are carried out on the stack s.

Stack operation	Stack contents	Return value
s.isEmpty()	[]	True
s.push('Cycle')		
s.push('Jump')		
s.push('Walk')		
s.isEmpty()		
s.peek()		
s.pop()		
s.pop()		

2. **[NYJC/Prelim/9597/P2/Q2]** A stack is to be implemented using an array of 20 elements.
 - (a) Describe an algorithm to remove an item from a stack and place it in a variable x.
 - (b) With the aid of examples, explain what nested functions or nested subroutines are.
 - (c) Explain with the aid of diagrams or otherwise, how a stack can be used by the operating system to process “nested functions” or “nested subroutines”.
 - (d) Outline the data attributes and member functions for a class stack abstract data type. You do not need to go into details as to how they will be implemented.
3. A stack is implemented using a linked list. Each node contains a data field and a next pointer. The top of the stack is pointed to by the variable head.
 - (a) Write pseudocode for the push and pop operations.
 - (b) Compare the use of a linked list versus an array to implement a stack. State one advantage and one disadvantage of the linked list implementation.
4. A student implemented the pop operation for a stack based on a linked list:

```

PROCEDURE pop()
  IF head == NULL THEN
    RETURN NULL
  END IF
  temp ← head
  head ← head.next
  RETURN temp
END PROCEDURE

```

- (a) Identify **two** potential issues in this implementation.
- (b) Suggest improvements to ensure robustness.

12. PROGRAMMING EXERCISE

1. Implement a stack using a Python list, where **the first element in the list is at the bottom of the stack and the last element is at the top of the stack.**

- Define a function `make_empty_stack()` that takes no arguments and returns an empty stack.
- Define a function `is_empty_stack(stack)` that returns True if the stack is empty and False otherwise.
- Define a function `push_stack(stack, item)` that takes in a stack and an item and pushes the item onto the top of the stack and returns the updated stack.
- Define a function `pop_stack(stack)` takes in a stack, removes the top item of that stack, and returns that top item. If the stack is empty, return None.
- Hence, define the function `match_brackets(expression)` which uses the stack to ensure the brackets of a mathematical expression are properly matched. The expression is a STRING input argument. The function returns True if all the brackets matches and False otherwise. There are 3 types of brackets: `()`, `[]`, and `{}`. For example, all brackets in “`{3+[(9+5)*10]}`” match but not “`[[9+5)*10]`”.

Use the following algorithm:

- Initialise the stack to empty.
 - Traverse the expression. For every character read
 - if open bracket then push onto stack, or
 - if close bracket then pop from stack and return False if it does not match.
 - At the end of traversal, return True if the stack is empty and False otherwise.
2. Implement a `Stack` class using array with the following properties and methods:

Class: <code>Stack</code>		
Properties		
Identifier	Data Type	Description
<code>Data</code>	<code>ARRAY[20] OF STRING</code>	Array to hold the stack elements. The array index starts at 1.
<code>Top</code>	<code>INTEGER</code>	Index position of the top element of the stack in the <code>Data</code> array.
Methods		
<code>Constructor</code>	<code>PROCEDURE</code>	Set up an empty stack and initialise value for <code>Top</code> .
<code>IsEmpty</code>	<code>FUNCTION RETURNS BOOLEAN</code>	Indicates whether any elements are stored in stack or not.
<code>IsFull</code>	<code>FUNCTION RETURNS BOOLEAN</code>	Indicates whether stack is full or not.
<code>Push</code>	<code>PROCEDURE</code>	Inserts data onto stack.
<code>Pop</code>	<code>FUNCTION</code>	Removes and returns the last inserted element from the stack.
<code>Peek</code>	<code>FUNCTION</code>	Returns the last inserted element from the stack without removing it.
<code>Size</code>	<code>FUNCTION</code>	Returns the number of elements stored in stack.
<code>Display</code>	<code>PROCEDURE</code>	Displays the content of the stack with top of stack clearly indicated.